Hormigas con instinto aprendido: réplica y mejora de DeepACO para el problema del agente viajero

Introducción y objetivos

El problema del agente viajero (TSP) consiste en visitar n ciudades una sola vez y volver al inicio con la menor distancia posible.

$$\min f(s) = \Sigma\, d(s_t, s_{t+1})$$

$x_{ij} \in \{0,1\}$: vale 1 si se viaja de la ciudad i a la j. Ciudades al azar en $[0,1]^2$. Cada ciudad se visita una vez y el recorrido forma un solo ciclo.

Inspiración biológica: las hormigas reales dejan feromona al caminar; los caminos cortos acumulan más y atraen a más hormigas, hasta que la colonia converge a la ruta más corta.

La optimización por colonia de hormigas (ACO) [2] imita esto con hormigas virtuales guiadas por la feromona τ y por una heurística η diseñada a mano. DeepACO [1] sustituye η por una red neuronal entrenada.

Objetivos

1. Replicar la Fig. 2 de Ye et al. [1].
2. Proponer y evaluar una mejora con búsqueda local 2-opt.
3. Valorar la replicabilidad y las dificultades.

Material y métodos

Instancia TSP: n ciudades en $[0,1]^2$

▼

η: red de grafos (12 capas) entrenada por refuerzo en DeepACO; 1/distancia en ACO clásico

▼

Las hormigas construyen rutas con probabilidad $\propto \tau^{\alpha} \cdot \eta^{\beta}$

▼

Mejora propuesta: 2-opt elimina los cruces de cada ruta

▼

Evaporación y refuerzo de τ; siguiente iteración

- Variantes de ACO: Ant System (AS), Elitist AS y MAX-MIN AS (MMAS).
- TSP20 y TSP100: 100 instancias, $T = 100$ iteraciones. TSP500: solo AS, 20 instancias, $T = 50$ (protocolo reducido por costo).
- $\alpha = \beta = 1$, evaporación 0.9 y 20 hormigas (50 en TSP500).
- 30 corridas por configuración (el artículo usó 3). Media, mediana, desviación estándar y prueba de Wilcoxon pareada.
- Código y modelos preentrenados de los autores [1] (Python y PyTorch).

Resultados de la réplica

La réplica reproduce los valores publicados con una diferencia menor a 0.2 % y DeepACO supera al ACO clásico en las 30 corridas de todas las configuraciones (Wilcoxon, $p < 10^{-8}$).

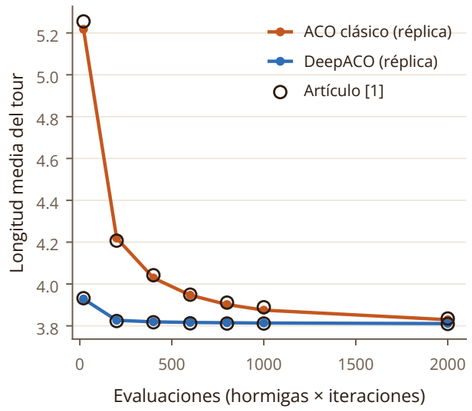


Figura 1. Réplica en TSP20 (Ant System): longitud del tour frente a evaluaciones. Líneas: promedio de 30 corridas; círculos: valores del artículo [1].

Caso	ACO	DeepACO
TSP20 · AS	3.829	**3.811**
TSP20 · Elitist	3.817	**3.808**
TSP20 · MMAS	4.281	**3.811**
TSP100 · AS	9.664	**8.234**
TSP100 · Elitist	9.439	**8.129**
TSP100 · MMAS	9.979	**8.276**
TSP500 · AS	**26.531**	**18.889**

Tabla 1. Longitud media del tour (30 corridas). Artículo en TSP20 · AS: ACO 3.835, DeepACO 3.809.

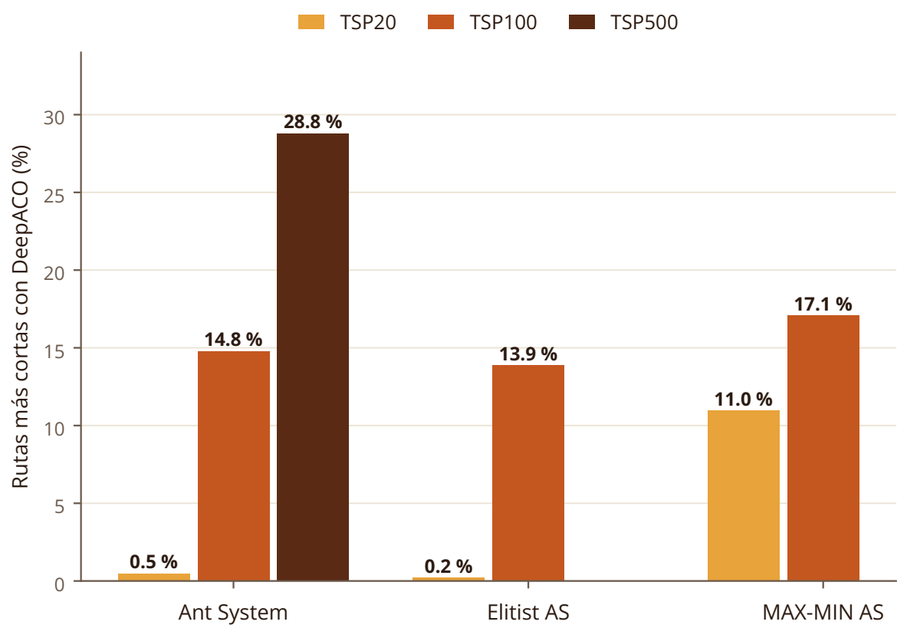


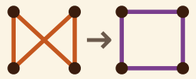
Figura 2. Reducción de la longitud del tour con DeepACO frente a ACO clásico, con las mismas hormigas e iteraciones (promedio de 30 corridas). TSP500: solo AS, 20 instancias, $T = 50$.

Con una sola iteración, DeepACO ya supera al ACO clásico tras 100 (TSP100). Su ventaja crece con el tamaño del problema y casi no depende de la variante de ACO.

Conclusiones

1. **DeepACO funciona.** Rutas 14–29 % más cortas que el ACO clásico en TSP100 y TSP500, con el mismo esfuerzo.
2. **Nuestra mejora funciona.** 2-opt acorta las rutas de DeepACO un 5.1 % adicional, sin reentrenar la red.
3. **La red acelera, pero explora menos.** Con 2-opt, el ACO clásico alcanza a DeepACO.
4. **La réplica es viable, con costos.** Diferencia menor a 0.2 % frente al artículo. Dificultades: hiperparámetros omitidos en el texto, dependencias obsoletas (PyTorch 1.7) y alto costo de cómputo (30 corridas frente a 3).

Resultados de la mejora: DeepACO + 2-opt



Qué hicimos: antes de depositar feromona, cada ruta pasa por 2-opt: si dos tramos se cruzan, se reconectan sin cruce y la ruta siempre se acorta.

Con 2-opt, DeepACO acorta 5.1 % su ruta (7.817 frente a 8.234 sin 2-opt; 30/30 corridas, $p < 10^{-8}$) y queda a 0.7 % del mejor DeepACO del artículo (7.763, con su búsqueda local NLS), sin reentrenar la red.

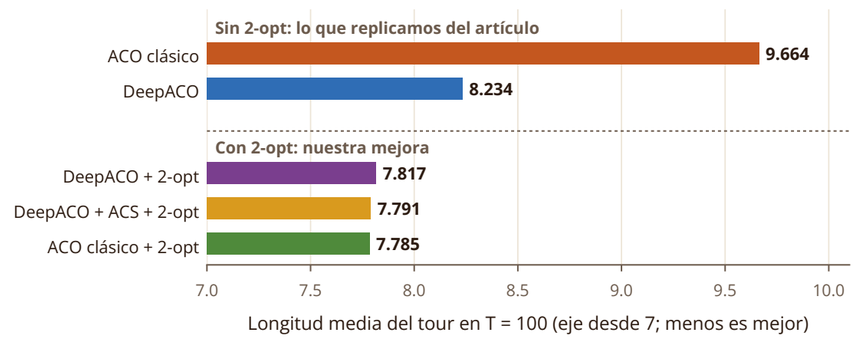


Figura 3. TSP100 (AS): longitud media del tour de cinco métodos, 30 corridas. Todos los métodos con 2-opt superan a DeepACO sin 2-opt en las 30 corridas (Wilcoxon, $p < 10^{-8}$). ACS: Ant Colony System [3], variante de ACO que refuerza solo la mejor ruta.

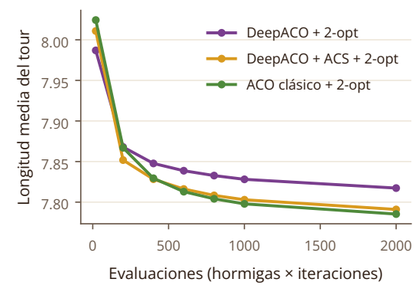


Figura 4. Longitud media del tour frente a evaluaciones (TSP100, 30 corridas).

Hallazgo. DeepACO arranca mejor, pero el ACO clásico lo alcanza hacia la iteración 10 y termina adelante (30 de 30 corridas).

Iteración	DeepACO + 2-opt	ACO + 2-opt
1	**7.987**	8.024
10	7.868	7.867
100	7.817	**7.785**

¿Por qué? La heurística aprendida es muy agresiva y reduce la exploración; con búsqueda local, explorar pesa más que el instinto, como reconocen los autores [1].

Una red neuronal le enseña a las hormigas a hallar rutas más cortas

28.8 %
rutas más cortas con DeepACO frente a ACO clásico (TSP500)

30/30
corridas ganadas por DeepACO en todas las configuraciones

< 0.2 %
de diferencia entre la réplica y los valores del artículo

−5.1 %
en la longitud de la ruta al sumar 2-opt a DeepACO (TSP100)

Integrantes

Rafael Santamaria Heredia
Grethel Hernandez Olivares

Tecnológico de Monterrey, Campus Guadalajara

Artículo replicado: Ye et al., NeurIPS 2023 [1]

Referencias

[1] Ye H, Wang J, Cao Z, Liang H, Li Y. DeepACO: neural-enhanced ant systems for combinatorial optimization. *Adv Neural Inf Process Syst.* 2023;36:43706-28. Disponible en: https://arxiv.org/abs/2309.14032

[2] Dorigo M, Stützle T. Ant colony optimization. Cambridge (MA): MIT Press; 2004.

[3] Dorigo M, Gambardella LM. Ant colony system: a cooperative learning approach to the traveling salesman problem. *IEEE Trans Evol Comput.* 1997;1(1):53-66.

[4] Santamaria Heredia R, Hernandez Olivares G. Deepaco-tsp-replication [Internet]. GitHub; 2026. Disponible en: https://github.com/RafaelHeredia12/Deepaco-tsp-replication

Repositorio

Código, datos y análisis completo [4]
github.com/RafaelHeredia12/Deepaco-tsp-replication